

# Deadlock Detection and Resource Management using Custom Linux System Calls

Nguyen Tien Dung (202417116)  
Nguyen Khac Khiem (202417142)  
Vu Sinh Hung (202417128)  
Bui Quang Minh (202417165)

## 1 Introduction

### 1.1 Background

A deadlock is a situation in an operating system where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process. Deadlock is a critical problem in operating systems because it permanently halts the execution of involved processes, leading to resource starvation and degraded system performance. Handling deadlocks involves prevention, avoidance, detection, and recovery.

### 1.2 Motivation

Understanding deadlocks requires knowledge of process synchronization, resource allocation, and graph theory. Therefore, implementing a deadlock detector inside the Linux kernel provides valuable experience with operating system internals. Working in kernel space allows us to interact directly with task states, process IDs, and wait queues, which is more authentic and challenging than simulating deadlocks in user space.

### 1.3 Objectives

The primary objectives of this project are to:

- Implement custom system calls inside the Linux Kernel.
- Simulate resource allocation for logical resources.
- Support sleeping processes by utilizing Linux wait queues.
- Detect deadlocks by constructing a Wait-For Graph and applying Depth-First Search (DFS).

## 2 Related Concepts

### 2.1 Deadlock Conditions

A deadlock can arise if and only if the following four conditions hold simultaneously in a system:

1. **Mutual Exclusion:** At least one resource must be held in a non-sharable mode.

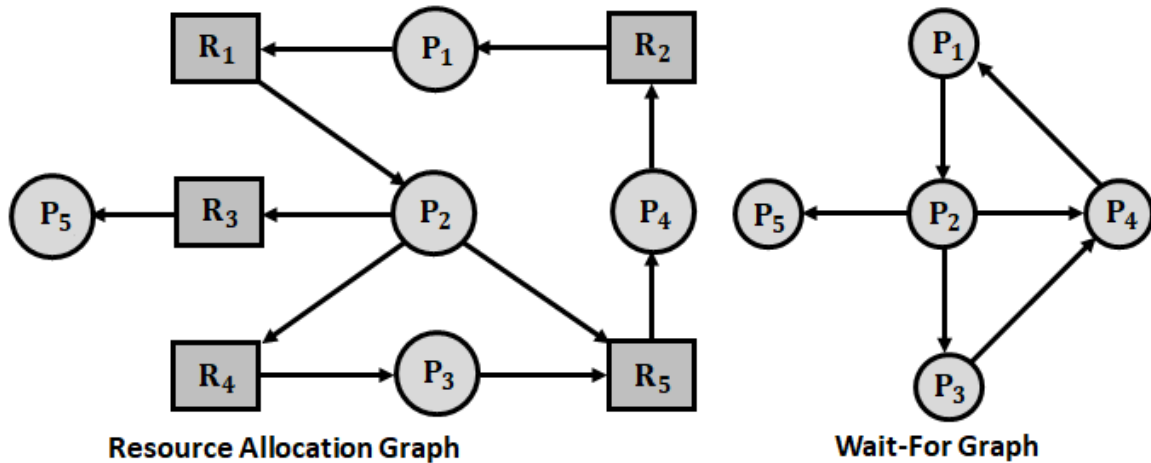
2. **Hold and Wait:** A process must be holding at least one resource and waiting to acquire additional resources held by other processes.
3. **No Preemption:** Resources cannot be forcibly taken from a process; they must be released voluntarily.
4. **Circular Wait:** There must exist a set of waiting processes  $\{P_1, P_2, \dots, P_n\}$  such that  $P_1$  is waiting for  $P_2$ ,  $P_2$  for  $P_3$ ,  $\dots$ , and  $P_n$  for  $P_1$ .

## 2.2 Wait-For Graph

A Wait-For Graph (WFG) is a directed graph used for deadlock detection.

- **Vertex:** Represents a process in the system.
- **Edge:** A directed edge  $P_i \rightarrow P_j$  indicates that process  $P_i$  is waiting for a resource held by process  $P_j$ .

Example:



$P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

If the wait-for graph contains a cycle, a deadlock exists.

## 2.3 Depth-First Search

Depth-First Search (DFS) is an algorithm for traversing or searching graph data structures. For cycle detection, it utilizes:

- **Visited set:** To track vertices that have already been evaluated.
- **Recursion stack:** To track vertices currently in the active traversal path. If a vertex is encountered that is already in the recursion stack, a cycle is detected.

## 3 System Design

### 3.1 Data Structures

#### 3.1.1 waiter\_node

```
1 struct waiter_node {  
2     pid_t pid;  
3     struct list_head list;  
4 };
```

This structure represents a process waiting for a resource. It stores the `pid` of the waiting process and a `list_head` to link it within a resource's list of waiting processes, effectively representing the outgoing edges in the Wait-For Graph.

#### 3.1.2 resource\_node

```
1 struct resource_node {  
2     int id;  
3     pid_t owner;  
4     wait_queue_head_t wait_queue;  
5     struct list_head waiters_list;  
6     struct list_head list;  
7 };
```

This structure represents a logical resource in the system:

- `id`: A unique integer identifier for the resource.
- `owner`: The `pid` of the process currently holding the resource (0 if free).
- `wait_queue`: A Linux wait queue allowing processes to efficiently sleep while waiting.
- `waiters_list`: A linked list of `waiter_node` representing processes currently blocked on this resource.
- `list`: A linked list node linking this resource into the global resource list.

### 3.2 Global Variables

- `resource_list`: The global linked list head containing all registered resources.
- `resource_mutex`: A global mutex to ensure thread-safe access to the resource list and state variables.

## 4 System Call Implementation

### 4.1 register\_resource()

**Purpose:** Registers a new logical resource in the kernel.

**Pseudo-code:**

```
1 Register_Resource(resource_id) {  
2     acquire the global mutex  
3  
4     if the resource already exists {  
5         release the mutex
```

```

6         return error
7     }
8     allocate and initialize a new resource
9
10    if allocation fails {
11        release the mutex
12        return error
13    }
14    insert the resource into the global resource list
15
16    release the global mutex
17
18    return success
19 }

```

## 4.2 request\_resource()

**Purpose:** Attempts to acquire a resource. If occupied, the process goes to sleep.

**Pseudo-code:**

```

1 Request_Resource(resource_id) {
2     acquire the global mutex
3     locate the resource
4
5     if resource does not exist {
6         release the mutex
7         return error
8     }
9
10    if resource is available (or already owned by us) {
11        grant ownership
12        release the mutex
13        return success
14    }
15
16    add the process to the waiting list
17
18    while (true) {
19        prepare to sleep
20
21        if resource is available {
22            acquire the resource
23            break
24        }
25
26        release the mutex
27        invoke schedule() // Process sleeps here
28        re-acquire the mutex
29    }
30
31    remove the process from the waiting list
32    release the mutex
33    return success
34 }

```

## 4.3 release\_resource()

**Purpose:** Releases ownership of a resource and wakes up waiting processes.

**Pseudo-code:**

```

1 Release_Resource(resource_id) {
2     acquire the global mutex
3     locate the resource
4
5     if resource does not exist or current process is not the owner {
6         release the mutex
7         return error
8     }
9
10    clear ownership
11    wake up all waiting processes
12
13    release the mutex
14    return success
15 }

```

#### 4.4 detect\_deadlock()

**Purpose:** Analyzes the current state of the system to find deadlocks.

**Pseudo-code:**

```

1 Detect_Deadlock() {
2     acquire the global mutex
3     build the set of participating processes
4
5     for each process p {
6         if p is unvisited {
7             if DFS(p) {
8                 release the mutex
9                 return true
10            }
11        }
12    }
13
14    release the mutex
15    return false
16 }

```

## 5 Deadlock Detection Algorithm

**Pseudo-code:**

```

1 DFS(u) {
2     mark visited
3     push u into recursion stack
4
5     for each neighbor v {
6         if v is unvisited {
7             if DFS(v)
8                 return true
9         } else if v is already in recursion stack {
10            return true // Cycle detected!
11        }
12    }
13
14    remove u from recursion stack
15    return false
16 }

```

**Complexity Analysis:** Although Depth-First Search itself has a complexity of  $O(V + E)$ , the current implementation dynamically constructs the wait-for graph by repeatedly traversing the resource list and waiter lists. Therefore, the overall complexity of deadlock detection is approximately  $O(V \times E)$ .

## 6 System Integration and Build Process

### 6.1 Development Environment

The project was developed and tested on Debian/Ubuntu with a modified Linux kernel. The custom system calls and supporting modules were implemented directly inside the kernel source tree.

### 6.2 System Call Integration

To integrate the deadlock detection subsystem into the Linux kernel, the following steps were performed:

1. **Create Subsystem Directory:** A new directory named `deadlock/` was created in the root of the kernel tree containing `syscalls.c`, `graph_algo.c`, and `deadlock.h`.
2. **Update Makefiles:** A Makefile was created in the `deadlock/` directory with the instruction `obj-y := syscalls.o graph_algo.o`. The main kernel Makefile was also modified to append `deadlock/` to the `core-y` variable.
3. **Register Syscalls:** The custom system calls were assigned unique numbers (e.g., 548 to 551) by appending them to the 64-bit syscall table located at `arch/x86/entry/syscalls/syscall_64`.
4. **Declare Prototypes:** The `asmlinkage` function prototypes for the four system calls were declared in the global header `include/linux/syscalls.h`.

After these modifications, the kernel source tree contained the complete deadlock detection subsystem.

### 6.3 Kernel Compilation and Installation

For Debian/Ubuntu systems, the necessary build dependencies must first be installed to compile the kernel successfully:

```
1 sudo apt update
2 sudo apt install build-essential libncurses-dev bison flex libssl-dev libelf
  -dev
```

After integrating the source files, the current OS configuration was copied, and the Linux kernel was rebuilt using:

```
1 cp /boot/config-$(uname -r) .config
2 make olddefconfig
3 make -j$(nproc)
4 sudo make modules_install
5 sudo make install
6 sudo update-grub
```

After rebooting, the system loaded the newly compiled kernel containing the custom system calls.

## 6.4 User-Space Testing

User programs accessed the custom system calls through the `syscall()` interface. For example,

```
1 syscall(__NR_register_resource, 1);
2 syscall(__NR_request_resource, 1);
3
4 ret = syscall(__NR_detect_deadlock);
5
6 syscall(__NR_release_resource, 1);
```

The return value of `detect_deadlock()` indicates whether a deadlock cycle exists among the currently active processes.

## 6.5 Testing Scenarios

Several scenarios were used to verify the correctness of the implementation:

- Resource acquisition without contention.
- Multiple processes waiting for the same resource.
- Two-process circular waiting.
- Three-process circular waiting.
- Recovery after resource release.

The experimental results confirmed that the system correctly manages resources and detects deadlocks through Wait-For Graph cycle analysis.

## 7 Linux Kernel Mechanisms Used

| Mechanism               | Purpose                                                                    |
|-------------------------|----------------------------------------------------------------------------|
| System Calls            | User-kernel interface                                                      |
| Linked List             | Resource management ( <code>struct list_head</code> )                      |
| <code>kmalloc()</code>  | Dynamic memory allocation in kernel space                                  |
| Mutex                   | Synchronization<br>( <code>mutex_lock</code> , <code>mutex_unlock</code> ) |
| Wait Queue              | Process sleeping ( <code>wait_queue_head_t</code> )                        |
| <code>schedule()</code> | Context switching and yielding CPU                                         |
| DFS                     | Cycle detection in Wait-For Graph                                          |

## 8 Experimental Results

To verify the deadlock detection subsystem, two user-space monitor programs were developed and executed to simulate different scenarios using `fork()`:

### 8.1 Scenario 1: Deadlock Occurrence (`demodeadlock.c`)

This program orchestrates a classic deadlock scenario:

1. **Initialization:** The monitor randomly generates and registers two logical resources,  $R_1$  and  $R_2$ , into the kernel.

2. **Process A:** A child process is spawned. It requests and acquires  $R_1$ , sleeps for 3 seconds, and then requests  $R_2$ .
3. **Process B:** A second child process is spawned concurrently. It requests and acquires  $R_2$ , sleeps for 3 seconds, and then requests  $R_1$ .

Due to the sleeping period, both processes acquire their first resource before attempting to acquire the second one. At this point, a circular wait is established: Process A is blocked waiting for  $R_2$  (held by B), and Process B is blocked waiting for  $R_1$  (held by A).

After a 5-second delay to ensure the system reaches a deadlock state, the parent monitor process invokes `syscall(SYS_detect_deadlock)`. The kernel successfully traverses the Wait-For Graph, detects the cycle, and returns 1. The program correctly reports:

Deadlock occurs!

## 8.2 Scenario 2: Safe System (`demonodeadlock.c`)

This program demonstrates a scenario without circular dependencies:

1. **Initialization:** The monitor randomly generates and registers a single logical resource,  $R$ .
2. **Process A:** A child process is spawned. It requests and acquires  $R$ , and then sleeps for 3 seconds.
3. **Process B:** A second child process is spawned concurrently. It also requests  $R$ . Since Process A already holds  $R$ , Process B goes to sleep, waiting for the resource.

In this case, Process B is waiting on Process A, but Process A is not waiting on any other process. The Wait-For Graph is linear and contains no cycles.

After a 5-second delay, the parent monitor process invokes `syscall(SYS_detect_deadlock)`. The kernel successfully determines that there are no cycles and returns 0. The program outputs:

The system is safe

In both scenarios, after detection, the monitor safely cleans up by sending a `SIGKILL` signal to both child processes.

## 9 Discussion

### Advantages:

- **Simple implementation:** Easy to integrate into the Linux kernel without modifying core subsystems.
- **Demonstrates OS concepts:** Provides a hands-on view of how an OS handles resources and synchronization.
- **Real process blocking:** Utilizing real wait queues ensures processes consume no CPU time while waiting.

### Limitations:

- **Single-instance resources:** The current model only supports resources with a single instance.



- **No deadlock recovery:** The system detects deadlocks but relies on the user-space monitor to kill processes.
- **Recursive DFS:** Recursion in the kernel can lead to stack overflow if the graph is extremely deep.
- **Fixed MAX\_PROCS:** The array sizes limit the number of processes analyzed simultaneously.

## 10 Future Work

The project can be expanded in several ways:

- **Deadlock recovery:** Implement kernel-level logic to automatically kill one process to break the cycle.
- **Multiple resource instances:** Modify the structures to support multiple instances of a resource, allowing multiple owners.
- **Timeout mechanism:** Abort waiting after a specified period to naturally prevent indefinite deadlocks.
- **Background deadlock monitor:** Implement a periodic detection kernel thread instead of relying on a user-space system call.
- **Victim selection policy:** Instead of killing a random process, choose the least expensive process to terminate based on CPU time or priority.

## 11 Conclusion

In this project, a custom resource management subsystem was implemented inside the Linux kernel. By combining wait queues, mutexes, linked lists, and graph cycle detection, the system successfully simulates resource allocation and detects deadlocks among processes. The project provides practical experience with Linux kernel programming and operating system concepts.

## References

- [1] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*, 10th Edition.
- [2] Robert Love. *Linux Kernel Development*, 3rd Edition.
- [3] Linux Kernel Documentation. <https://www.kernel.org/doc/html/latest/>